

Aula 02 - Elementos de um Sistema IoT

Descrição: Estudo dos elementos que compõem um sistema IoT (dispositivos, conectividade, dados, processamento) e painel temático sobre aplicações industriais, com elaboração do mapa conceitual de arquitetura.

1. Camada do Mundo Físico: Sensoriamento e Atuação

Esta camada é a porta de entrada e saída do sistema, responsável por traduzir grandezas físicas em sinais elétricos (sensoriamento) e ações mecânicas ou térmicas de volta ao ambiente (atuação).

1.1 Sensores (Entrada de Dados)

Os sensores são dispositivos essenciais que detectam fenômenos físicos ou químicos no ambiente — como temperatura, luminosidade, pressão e movimento — e os convertem em sinais elétricos legíveis por microcontroladores e computadores. Eles atuam como a "porta de entrada" dos dados no mundo digital, viabilizando o monitoramento em tempo real, a automação industrial e o funcionamento de redes de Internet das Coisas (IoT).

Aspectos Fundamentais dos Sensores

- **Natureza do Sinal:** Podem gerar saídas **analógicas** (variações contínuas de tensão ou resistência) ou **digitais** (dados codificados diretamente em bits).
- **Tratamento do Sinal:** Sinais analógicos dependem de conversores analógico-digitais (ADC) e circuitos de condicionamento (como divisores de tensão) para serem processados.
- **Exemplos Clássicos:** Fotorresistores de luz (LDR), medidores de distância por ondas sonoras (ultrassônicos) e sensores de temperatura (analógicos como o LM35 ou digitais como o DS18B20).
- **Papel na IoT:** Formam a base da camada do mundo físico, capturando eventos do ambiente real para alimentar a tomada de decisão em sistemas inteligentes.

Tipos de sensores

- **LDR (Light Dependent Resistor):** É um fotorresistor feito de sulfeto de cádmio (CdS). Na ausência de luz (fótons), apresenta uma resistência elétrica muito alta (centenas de kΩ); sob iluminação, os elétrons são liberados e sua resistência cai drasticamente. Para ler essa variação no Arduino, monta-se um circuito **divisor de tensão** com um resistor fixo (normalmente de 10 kΩ) para gerar uma tensão de saída (Vout) proporcional à luminosidade.
- **Sensor Ultrassônico (HC-SR04):** Usado para medição de distância de precisão (limite prático de 2 cm a 200 cm). O processador local envia um pulso elétrico no pino *Trigger* por **10 microssegundos**, disparando a emissão de **8 pulsos sonoros de 40 kHz**. O pino *Echo* entra em nível alto (HIGH) e o microcontrolador mede em microssegundos o tempo decorrido até o sinal sonoro retornar ao receptor. A distância em centímetros é calculada usando a velocidade do som no ar (≈ 340 m/s), resultando na fórmula de calibração: $distância = duração / 58$.
- **Sensores de Temperatura:**
 - **LM35 (Analogico):** Fornece uma tensão de saída linearmente proporcional à temperatura em Celsius (10mV/°C). Em bancada, para otimizar a resolução da leitura analógica no conversor ADC de 10 bits do Arduino (0 a 1023), pode-se configurar a **voltagem de referência interna** de 1,1V (`analogReference(INTERNAL)`), o que eleva a precisão do cálculo de conversão, já que a escala de leitura se adequa à faixa de saída máxima do LM35.
 - **DS18B20 (Digital):** Comunica-se por protocolo **1-Wire (1 Fio)**, permitindo que vários sensores de temperatura sejam ligados em cascata (encadeados) sob um único pino digital do microcontrolador. Cada sensor possui um código de identificação de hardware (ID) de 64 bits único de fábrica. Sua resolução é configurável de 9 a 12 bits, variando o tempo de conversão de 93,75 ms a 750 ms.

1.2 Atuadores (Saída de Ação)

Os atuadores são componentes que convertem sinais lógicos ou elétricos em ações mecânicas, magnéticas ou outras formas de energia no ambiente físico. Eles representam a camada de saída de um sistema IoT, permitindo que as decisões do microcontrolador executem modificações diretas no mundo real.

Aspectos Fundamentais dos Atuadores

- **Transformação de Energia:** Convertem a inteligência de software em trabalho físico, como rotação, deslocamento ou geração de torque.
- **Interface e Amplificação de Potência:** Como costumam exigir correntes elevadas (frequentemente 500 mA ou mais), não podem ser alimentados diretamente pelas portas do microcontrolador (limitadas a 40 mA), necessitando de transistores ou drivers intermediários.
- **Exemplos Clássicos:** Motores de corrente contínua (CC) para rotação contínua, Pontes H para controle de sentido/velocidade e servomotores para posicionamento angular preciso.
- **Papel na IoT:** Fecham a malha de controle do sistema, transformando os dados capturados pelos sensores e processados na borda em ações práticas e automatizadas no ambiente.

Tipos de Atuadores

- **Motores de Corrente Contínua (CC):** Motores de CC exigem correntes elétricas elevadas para operar (geralmente de 500 mA ou mais). Como as portas digitais do Arduino fornecem uma corrente limite de segurança de apenas **40 mA por pino**, tentar conectar um motor diretamente à porta causará danos permanentes ao chip. Para isolar e amplificar a corrente, empregam-se **transistores bipolares NPN (como TIP120, BD135, BD137 ou BD139)** atuando como chaves eletrônicas acionadas pelas portas do Arduino.
- **Pontes H (Drivers Integrados):** Para controlar a velocidade e a rotação bidirecional de motores CC, utilizam-se circuitos de Ponte H. ICs populares como o **L293D ou o SN754410** encapsulam pontes H duplas e possuem diodos internos de proteção contra surtos eletromagnéticos reversos (força eletromotriz inversa) gerados pelas bobinas indutivas dos motores ao serem desligados.
- **Servomotores:** Contêm em um único bloco um motor CC, engrenagens, um potenciômetro de feedback de posição e um circuito de controle. Eles posicionam o seu braço de saída em ângulos específicos (geralmente até 180°, respondendo a sinais modulados por largura de pulso).

1.3 Condicionamento de Sinais, Qualidade e Segurança de Bancada

O condicionamento de sinais e a segurança de bancada garantem a integridade dos circuitos eletrônicos e a confiabilidade das leituras em um sistema embarcado. Essas práticas eliminam ruídos elétricos, estabilizam estados lógicos e protegem os microcontroladores contra surtos de tensão e sobreaquecimento.

Na montagem física em protoboard, o respeito às normas de qualidade evita a queima de componentes:

- **Resistores de Pull-up e Pull-down:** Essenciais para chaves digitais e botões (*push-buttons*). Sem um resistor conectado à referência, o pino do microcontrolador fica em **estado flutuante (floating)**, captando ruídos eletromagnéticos do ar e gerando falsos acionamentos. O resistor de *pull-down* garante o nível lógico baixo (0 V) quando o botão está solto; ao pressioná-lo, a tensão vai para 5 V (HIGH). O Arduino possui resistores de *pull-up* internos (20 kΩ a 50 kΩ) que podem ser ativados via software (`pinMode(pino, INPUT_PULLUP)`).
- **Diodos de Proteção (como o 1N4001):** Devem ser conectados em paralelo reverso com cargas indutivas (motores e relés) para proteger os transistores e o microcontrolador de picos de alta tensão gerados pelo desligamento da bobina.
- **Dissipadores de Calor:** Componentes como o L293D e transistores de potência aquecem significativamente sob cargas contínuas e exigem o uso de dissipadores metálicos térmicos para evitar falhas por sobreaquecimento.

2. Camada de Processamento Local: Microcontroladores e Programação Embarcada

Esta é a camada de **Inteligência de Borda (Edge Computing)**, onde o processamento local filtra, armazena temporariamente ou toma decisões rápidas próximas aos sensores para evitar tráfego de dados massivos e redundantes para a rede.

2.1 O Microcontrolador (Hardware de Controle)

Um microcontrolador é um sistema de computação em chip único (*system-on-a-chip*) composto por uma unidade central de processamento (CPU), memória RAM para variáveis de trabalho (volátil), memória Flash para gravação do código compilado (não volátil), EEPROM para dados persistentes de calibração que sobrevivem à falta de energia, e portas de Entrada/Saída (I/O):

- **Arduino Uno (ATmega328):** Possui CPU AVR de 8 bits, clock de 16 MHz, **2 kB de memória SRAM** (RAM estática), **32 kB de memória Flash** e 1 kB de EEPROM. É um hardware excelente para fins didáticos e controle básico de entradas e saídas digitais e analógicas.
- **Arduino Mega 2560 (ATmega2560):** Indicado para projetos complexos que exigem alta quantidade de conexões físicas. Dispõe de **54 portas digitais** (15 com suporte a PWM), **16 entradas analógicas**, 256 kB de memória Flash e 8 kB de SRAM.
- **Classes de Dispositivos Limitados:** Conforme a especificação IETF RFC 7228, o hardware de borda na IoT é categorizado por limitações de recursos elétricos e computacionais:
 - **Classe 0:** Dispositivos altamente limitados (RAM << 10 kB, Flash << 100 kB), que não possuem capacidade para executar de forma nativa a pilha TCP/IP padrão, exigindo proxies ou gateways para se comunicarem na Internet.
 - **Classe 1:** Dispositivos intermediários (RAM ≈ 10 kB, Flash ≈ 100 kB), capazes de rodar pilhas de protocolos altamente otimizados e comprimidos (como 6LoWPAN) sem o auxílio constante de um gateway.
 - **Classe 2:** Dispositivos menos limitados (RAM ≈ 50 kB, Flash ≈ 250 kB), que executam a pilha IP padrão de forma completa.

2.2 Programação Embarcada na Prática (C/C++ e Lógica Embarcada)

Os sketches de microcontroladores são desenvolvidos em dialeto da linguagem C/C++. O desenvolvimento exige atenção restrita ao limite físico da memória do hardware:

- **Funções `setup()` e `loop()`:** Essenciais em qualquer código embarcado. A função `setup()` é executada apenas uma vez ao iniciar o sistema para configurar estados dos pinos e portas; a função `loop()` executa as rotinas sequencialmente e infinitamente enquanto houver energia.
- **Eficiência de Memória nos Tipos de Dados:** No desenvolvimento embarcado, cada byte é valioso. Salvar o número de uma porta digital como `int` (inteiro) consome **2 bytes** na SRAM (faixa de -32.768 a 32.767). Como os pinos do Arduino Uno variam apenas de 0 a 13, economiza-se memória RAM declarando os pinos como tipo `byte` ou `char` (que consomem apenas **1 byte** e armazenam valores de 0 a 255).
- **Função `millis()` versus `delay()`:** A utilização da função `delay()` congela a execução da CPU do microcontrolador por milissegundos definidos. Em aplicações de IoT, onde o processador deve receber comandos pela porta serial, monitorar sensores em tempo real e atuar de forma concorrente, o congelamento por `delay()` inviabiliza o sistema. A alternativa correta é monitorar o tempo de processamento por meio da função `millis()`, que retorna um inteiro longo sem sinal (`unsigned long`, de 4 bytes) que armazena os milissegundos transcorridos desde que a placa foi ligada. O estado do sensor ou das transmissões de rede é então verificado comparando-se marcas temporais de processamento sem pausar a CPU.

3. Camada de Conectividade de Hardware e Infraestrutura de Redes

A conectividade é o sistema vascular da IoT, transportando informações entre os dispositivos inteligentes de borda, gateways e servidores centrais.

3.1 Tecnologias de Hardware Sem Fio (Alcance e Aplicação)

- **IEEE 802.15.4 (ZigBee):** Tecnologia sem fio focada em baixo consumo de energia, baixa complexidade e custo acessível de hardware. Opera prioritariamente nas faixas ISM (frequência livre) de 2,4 GHz, 915 MHz (Américas) e 868 MHz (Europa), com taxas de vazão de até 250 kbps. Suporta redes em topologia estrela, árvore de clusters ou malha (mesh), sendo ideal para redes de sensores pessoais ou industriais locais (WPANs).
- **Bluetooth Low Energy (BLE):** Diferente do Bluetooth clássico (focado em taxas de vazão elevadas para áudio e arquivos), o BLE (Bluetooth 4.0 ou superior) é otimizado para baixíssimo consumo, operando por longos meses com pequenas baterias em formato de moeda. Tem excelente penetração em dispositivos móveis modernos (smartphones e tablets) e suporta topologias estrela e mesh.
- **LoRaWAN (Low Power Wide Area Network):** Tecnologia de LPWAN projetada para dispositivos de baixo consumo que se comunicam a longas distâncias. Utiliza modulação por espalhamento espectral de chirp (Chirp Spread Spectrum), oferecendo uma sensibilidade de rádio impressionante e alto orçamento de link (*link budget*). Consegue penetrar paredes e atingir alcances práticos de 2 a 5 km em regiões urbanas densas e até 45 km em áreas rurais de visada direta, com taxas de transmissão de 300 bps a 50 kbps.
- **Wi-Fi (IEEE 802.11):** Solução tradicional baseada em alta taxa de transferência (vazão de megabits a gigabits) e longo alcance em redes locais de topologia estrela. A desvantagem principal reside em seu consumo energético altíssimo, o que inviabiliza seu uso em sensores de IoT de campo alimentados por baterias. O padrão recente **IEEE 802.11ah (Wi-Fi HaLow)**, operando abaixo de 1 GHz, visa contornar essa falha oferecendo comunicação de baixo consumo e longo alcance para IoT.

3.2 Redes de Computadores e a Pilha IP

- **Adaptação com 6LoWPAN:** O protocolo IPv6 padrão, com seu endereçamento de 128 bits e cabeçalho base de 40 bytes, é muito pesado para quadros de transmissão de redes limitadas (como o limite físico de carga de 127 bytes do IEEE 802.15.4). A camada de adaptação **6LoWPAN** atua entre a camada física/enlace e a camada de rede IP para realizar a compressão inteligente de cabeçalhos IPv6, fragmentação e montagem de pacotes, e encaminhamento I2 (*mesh under*), permitindo que dispositivos de Classe 1 acessem a Internet nativamente.
- **Roteamento com RPL:** O RPL (*Routing Protocol for Low-Power and Lossy Networks*) é o protocolo padrão da IETF de vetor de distância para rotear mensagens em redes IPv6 instáveis e limitadas (LLNs).
 - O RPL cria uma árvore de roteamento dinâmica chamada **DODAG** (*Destination-Oriented Directed Acyclic Graph*) que converge para o roteador de borda (raiz).
 - Utiliza mensagens **DIO** (*DODAG Information Object*) emitidas pela raiz e nós pais para propagar informações de conexão e classificar a distância lógica de cada dispositivo em relação à raiz através do cálculo do **Rank**.
 - Mensagens **DIS** (*DODAG Information Solicitation*) são usadas de forma ativa por nós que se ligaram recentemente para descobrir redes vizinhas ativas.
 - Mensagens **DAO** (*DODAG Advertisement Object*) são enviadas pelos nós das folhas em direção ascendente (para os pais) para informar as rotas e construir as tabelas de roteamento descendentes na árvore, viabilizando o tráfego bidirecional ponto-a-ponto.

4. Camada de Software, Protocolos e Integração Web

Esta camada é a ponte que integra o dado gerado na borda à nuvem e aos servidores, permitindo que a inteligência de negócios consuma os dados de sensores de forma amigável.

4.1 Protocolos de Aplicação IoT

As aplicações tradicionais da Web usam o protocolo HTTP sobre conexões TCP. No entanto, a sobrecarga de mensagens necessárias para estabelecer uma conexão TCP síncrona (com o handshake de três vias) torna o HTTP ineficiente para dispositivos limitados e links de comunicação sujeitos a perdas. Os protocolos de aplicação de IoT são projetados para serem extremamente leves:

- **CoAP (Constrained Application Protocol):** É um protocolo de comunicação binário e assíncrono desenvolvido pela IETF. Ele adota a arquitetura **RESTful** (fazendo uso de requisições GET, POST, PUT, DELETE sobre URIs), mas executa de forma muito mais leve sobre o protocolo de transporte de datagramas **UDP**, eliminando a necessidade de handshakes complexos do TCP. O CoAP possui um recurso nativo chamado **Observe**, no qual um cliente pode registrar seu interesse em uma URI do sensor e o servidor CoAP envia notificações automáticas apenas quando houver mudanças no estado físico, poupando largura de banda.
- **MQTT (Message Queue Telemetry Transport):** É um protocolo binário leve que segue a arquitetura de **Publicação/Assinatura (Publish/Subscribe)** e executa sobre o transporte TCP para garantir entrega e ordenamento de mensagens. A comunicação é totalmente mediada por um servidor central chamado **Broker**:
 - Os **Publishers** (como microcontroladores conectados a sensores) publicam dados sob um endereço estruturado de tópicos (por exemplo, *casa/quarto/temperatura*).
 - Os **Subscribers** (como dashboards ou bancos de dados na nuvem) assinam os tópicos de interesse.
 - O **Broker** gerencia as conexões, filtra as mensagens por tópicos e distribui os dados aos assinantes de forma rápida e assíncrona, desacoplando o emissor do receptor.

4.2 oneM2M e ETSI M2M (Padrões de Integração e Abstração)

O desenvolvimento clássico de sistemas IoT sofre com o acoplamento excessivo de software com o hardware proprietário. Substituir um sensor danificado de um fabricante por outro exige a reescrita do código da aplicação, gerando alto custo operacional.

Os consórcios de padronização criaram middlewares de abstração para viabilizar o modelo "qualquer aplicação se comunica com qualquer dispositivo":

- O padrão **oneM2M** propõe uma arquitetura que define a **Common Services Entity (CSE)**. A CSE funciona como uma camada de software intermediária (*middleware*) que roda em servidores na nuvem, gateways e dispositivos robustos, provendo funções comuns padronizadas de gerenciamento de dados,criptografia e segurança, descoberta de recursos físicos e lógica de gerenciamento de hardware por meio de APIs abstratas e padronizadas, reduzindo a complexidade de desenvolvimento de soluções IoT de ponta a ponta.

Atividade de Pesquisa Complementar

Título: *Dimensionamento de Arquitetura IoT para Cenários de Aplicação Real*

Objetivo: Aplicar os conceitos das quatro camadas apresentadas na aula para projetar a infraestrutura técnica de uma solução de IoT viável mercado.

Roteiro da Atividade:

1. Escolha de um Cenário Prático:

Individual (ou dupla) deve selecionar **um** dos cenários abaixo:

- **Agrotech:** Monitoramento de umidade do solo e acionamento de irrigação em fazendas conectadas.
- **Cidades Inteligentes:** Gestão de iluminação pública e detecção de vagas em estacionamentos.
- **Indústria 4.0:** Preditiva de falhas por temperatura e vibração em motores de esteiras fabris.

2. Mapeamento das 4 Camadas:

- **Camada Física:** Selecionar sensores e atuadores para o cenário. Indicar quais circuitos de proteção ou condicionamento (e.g., diodos, *pull-up/down*, divisores) serão necessários.
- **Processamento na Borda:** Definir a classe do dispositivo conforme a RFC 7228 (Classe 0, 1 ou 2) e explicar como a lógica de código evita travamentos da CPU usando temporização assíncrona.
- **Conectividade:** Comparar duas tecnologias de rede (ex.: LoRaWAN vs BLE ou ZigBee vs Wi-Fi) avaliando raio de alcance, autonomia de bateria e topologia de rede.
- **Aplicação e Software:** Escolher entre MQTT e CoAP, justificando a decisão com base na camada de transporte (TCP vs UDP) e desenhando a estrutura do tópico ou URI dos dados.

3. Entregável:

Relatório técnico (2 a 3 páginas) contendo o diagrama de blocos das 4 camadas e uma matriz comparativa com as justificativas técnicas das escolhas adotadas.